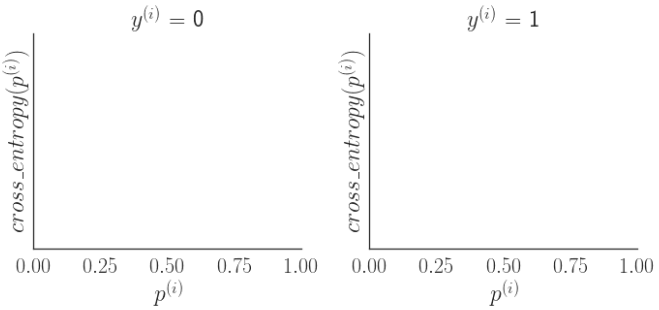
# 11. Regresión Logística (Clasificación)

**Ejercicio 11.1.** Para clasificación a través de la técnica de regresión logística, suele utilizarse la función "Binary Cross-Entropy" para definir el error de una predicción en particular. Este error se puede escribir como:

$$\text{Binary_CE}(\mathbf{y}^{(i)}, \mathbf{p}^{(i)}) = \begin{cases} -\log(\mathbf{p}^{(i)}) & \text{si } \mathbf{y}^{(i)} = 1 \\ -\log(1 - \mathbf{p}^{(i)}) & \text{si } \mathbf{y}^{(i)} = 0 \end{cases}$$

(a) ¿A qué hacen referencias las variables $\mathbf{y}^{(i)}$ y $\mathbf{p}^{(i)}$ en este cálculo?

(b) Completar los gráficos dibujando el costo asociado al error según la clase original de la instancia:




(c) Explicar con sus palabras por qué es bueno que este costo esté cerca de cero y en qué caso se acerca a infinito.

(d) Expresar la fórmula completa del costo asociado a un conjunto de datos $\mathbf{X}$ y sus etiquetas $\mathbf{y}$.

**Ejercicio 11.2.** ¿Cuál es el valor esperado de predicción de un modelo de regresión logística que utiliza regularización L2 (Ridge) con $\lambda$ tendiendo a infinito?

**Ejercicio 11.3.** Los gráficos de la Figura 3 han sido generados mediante la función sigmoidea: $sigm(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$. Para cada uno, $z^{(i)}$ fue calculado utilizando distintos $w_0$ y $w_1$ siguiendo la fórmula $z^{(i)} = w_0 + w_1 * x_1^{(i)}$.

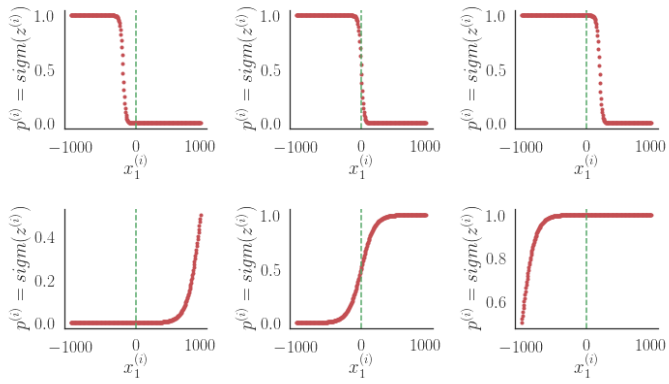



Figura 8: Sigmoideas

1. Determinar en qué dibujos $w_1$ es positivo y en cuáles negativo. ¿En qué afecta el signo de los distintos $w_s$ en el caso de instancias multidimensionales?
2. Por fila, ordenar los dibujos según su valor de $w_0$.

**Ejercicio 11.4.**

1. Escribir el pseudocódigo de la función "Descenso por gradiente" para el caso de regresión logística, comentando brevemente qué se espera de cada argumento (junto a su tipo).
2. Explicar cómo se obtiene el gradiente de la función a minimizar.
3. Escribir el pseudocódigo de mini-batch gradient descent.

# 12. Redes Neuronales (FNNs)

**Ejercicio 12.1.** Verdadero o Falso (justificar en cada caso):

(a) Un perceptrón simple con función de activación sigmoidea entrenado con entropía cruzada binaria es equivalente a un modelo de regresión logística.

(b) Un perceptrón simple con función de activación lineal define la misma familia de funciones que una regresión lineal.

(c) En un problema de regresión de $\mathbb{R}^p \to \mathbb{R}^q$, una red sin capas ocultas, con activación lineal y $q$ neuronas de salida, entrenada con error cuadrático medio, aprendería los mismos pesos que entrenar $q$ regresiones lineales independientes (una por cada salida). Ejemplo: predecir no sólo el valor de una casa sino también sus metros cuadrados según $p$ atributos.

(d) En un problema de regresión de $\mathbb{R}^p \to \mathbb{R}^q$, una red con capas ocultas con $q$ neuronas de salida aprendería los mismos pesos que entrenar $q$ redes neuronales independientes (con la misma arquitectura pero sólo 1 neurona de salida cada una).

**Ejercicio 12.2.** Construir a mano un perceptrón simple que resuelva el operador lógico AND: dadas dos variables $X_1$ y $X_2$, devuelve *True* o *False*. En las variables de entrada, interpretar $X_i = 1$ como *True* y $X_i = 0$ como *False*. Ídem para los operadores OR, NOR y NAND. ¿Es posible construir un perceptrón simple que resuelva XOR? Justificar (sugerencia: graficar las cuatro combinaciones de entrada en el plano $X_1$–$X_2$).

**Ejercicio 12.3.** Demostrar que una red neuronal con una neurona de salida, con función de activación lineal en todas las capas salvo la última, y activación sigmoidea en la última capa define la misma familia de funciones que una regresión logística. ¿Tiene sentido utilizar una red con muchas capas en este caso? ¿Qué desventajas prácticas (en cantidad de parámetros y en la optimización) tiene entrenar esta red profunda frente a entrenar directamente una regresión logística?

**Ejercicio 12.4. Cantidad de parámetros de una red neuronal**

(a) Sea una red neuronal densa con *biases* con una capa de entrada con 3 neuronas, una capa oculta con 4 neuronas y una capa de salida con 2 neuronas. Realizar un diagrama de dicha red y calcular la cantidad de pesos (incluyendo los *biases*) en esta red neuronal.

(b) Sea una red neuronal densa con $M$ capas ocultas, donde la $i$-ésima capa oculta tiene $N_i$ neuronas ($i = 1, 2, \ldots, M$), una capa de entrada con $I$ neuronas y una capa de salida con $O$ neuronas. Calcular la cantidad total de pesos (incluyendo los *biases*) en esta red neuronal.

(c) Suponer que cada capa oculta tiene una cantidad fija de $N_h$ neuronas para todas las capas. Determinar cómo crece la cantidad total de pesos en la red en términos de $M$, es decir, determinar la "*complejidad del modelo*" en términos de la cantidad de capas.

**Ejercicio 12.5.** Dada una red neuronal con la siguiente arquitectura, en donde $X$ es la matriz de entrada (una instancia por fila); $W^{[l]}$, la matriz de pesos que conecta la capa $l-1$ con la capa $l$; y $A^{[l]}$, las activaciones de la capa $l$:

$$X \in \mathbb{R}^{5\times 3}, \quad W^{[1]} \in \mathbb{R}^{*\times 2}, \quad A^{[1]} \in \mathbb{R}^{*\times *}, \quad W^{[2]} \in \mathbb{R}^{*\times 1}, \quad A^{[2]} \in \mathbb{R}^{*\times *}, \quad W^{[3]} \in \mathbb{R}^{*\times 2}, \quad Y \in \mathbb{R}^{*\times *}$$

(a) Escribir la fórmula denotada por esta red. Es decir, $Y = \ldots$ suponiendo funciones de activación $g_i$ para toda capa intermedia, y $g_o$ para la salida. Escribir dos versiones, una en la que los términos de bias están explícitos, una en la que no. Para lo segundo, utilizar la notación $M'$, que denota la matriz $M$ con una columna de unos agregada en primer lugar (como $X'$ en clase).

(b) Completar los valores faltantes denotados con asteriscos, considerando la versión con los términos de bias explícitos (es decir, sin la columna de unos en $X$ ni en las $A^{[l]}$).

(c) Dibujar el esquema de la red neuronal.

**Ejercicio 12.6. Experimentos en el playground de TensorFlow** (`https://playground.tensorflow.org`).

(a) Elegir el tercer dataset, que tiene dos grupos de puntos bien separados. Experimentar con diferentes configuraciones de capas ocultas, nodos y atributos, y estudiar el comportamiento de cada parte de la red durante el entrenamiento. Por ejemplo, usar:

- un solo atributo ($X_1$) y una capa con un nodo;
- un solo atributo ($X_1$) y dos o más capas con dos o más nodos;

- dos atributos ($X_1$, $X_2$) y una sola capa con un nodo; etc.

(b) Usando sólo los atributos $X_1$ y $X_2$, construir una configuración mínima (en cantidad de capas y de nodos por capa) de un perceptrón multicapa que resuelva el operador lógico XOR. Usar el segundo dataset.

(c) Usando sólo los atributos $X_1$ y $X_2$, construir un perceptrón multicapa que pueda aprender los otros dos problemas no linealmente separables: el círculo azul rodeado de amarillo (fácil) y la doble espiral (difícil).

(d) Estudiar cómo impacta en el aprendizaje de los ítems (b) y (c) la inclusión de otros atributos (por ejemplo, $\sin(X_1)$), así como la elección de distintas funciones de activación (lineal, tanh, sigmoid).

**Ejercicio 12.7. Backpropagation a mano**
Una red con un solo atributo $x$, una neurona oculta con activación sigmoidea $\sigma$ y una neurona de salida lineal:

$$h = \sigma(w_1 x + b_1) \qquad \hat{y} = w_2\, h + b_2 \qquad J = (\hat{y} - y)^2$$

Se quiere el gradiente de $J$ respecto de los parámetros en el punto $w_1 = 0.5$, $b_1 = -1$, $w_2 = 2$, $b_2 = 0.5$, para la instancia $x = 2$, $y = 1$. Dato: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$.

(a) Dibujar el grafo computacional de $J$, distinguiendo qué nodos de entrada se actualizan durante el entrenamiento y cuáles permanecen fijos.

(b) **Forward:** calcular el valor de cada nodo.

(c) **Derivadas locales:** calcular la derivada de la salida de cada nodo respecto de cada una de sus entradas, en el punto.

(d) **Backward:** calcular $\frac{\partial J}{\partial w_1}$, $\frac{\partial J}{\partial b_1}$, $\frac{\partial J}{\partial w_2}$ y $\frac{\partial J}{\partial b_2}$, recorriendo el grafo desde $J$ hacia las entradas. Calcular también $\frac{\partial J}{\partial x}$.

(e) Dar un paso de descenso por gradiente con $\alpha = 0.1$ y verificar que $J$ disminuye (dato: $\sigma(-0.25) \approx 0.438$).

(f) Si hubiera dos instancias y el costo fuera el promedio de los $J^{(i)}$, ¿cómo cambiaría el grafo? ¿Cómo se combinan los aportes de cada instancia en el gradiente de $w_1$?

**Ejercicio 12.8. Backpropagation (forma general)**
En este ejercicio, consideraremos una red neuronal con las siguientes definiciones:

- La entrada $z$ de una neurona $j$ en la capa $l$ está dada por:

$$z_j^{[l]} = \sum_i w_{i,j}^{[l]} a_i^{[l-1]} + b_j^{[l]}$$

  donde $a_i^{[l]} = \sigma(z_i^{[l]})$, y $\sigma$ es la función de activación. Aquí, $i$ representa los índices de las neuronas de la capa anterior. Además $w_{i,j}^{[l]}$ representa el peso desde la neurona $i$ de la capa $l-1$ a la neurona $j$ de la capa $l$.

- La función de costo $J$ para la red neuronal está definida como:

$$J = \sum_i \frac{1}{2}(y_i - a_i^{[L]})^2$$

  donde $L$ es la última capa de la red neuronal e $y_i$ es el valor esperado de la salida $i$.

- Para cada capa $l$ y cada neurona $j$ definimos

$$\delta_j^{[l]} = \frac{\partial J}{\partial z_j^{[l]}}$$

Utilizando estas definiciones, resolver los siguientes problemas:

(a) Demostrar que:

$$\delta_j^{[L]} = (a_j^{[L]} - y_j)\,\sigma'(z_j^{[L]})$$

(b) Demostrar que:

$$\delta_j^{[l]} = \sigma'(z_j^{[l]}) \sum_k \delta_k^{[l+1]}\, w_{j,k}^{[l+1]}$$

Sugerencia: usar la siguiente igualdad:

$$\delta_j^{[l]} = \sum_k \frac{\partial J}{\partial z_k^{[l+1]}} \frac{\partial z_k^{[l+1]}}{\partial z_j^{[l]}}$$

donde $k$ recorre las neuronas de la capa $l+1$.

(c) Demostrar que:

$$\frac{\partial J}{\partial w_{i,j}^{[l]}} = \delta_j^{[l]}\, a_i^{[l-1]}$$

(d) Demostrar que:

$$\frac{\partial J}{\partial b_j^{[l]}} = \delta_j^{[l]}$$

(e) Conceptualmente, ¿qué mide $\delta_j^{[l]}$ y qué ocurre con la actualización de los pesos y el sesgo de la neurona $j$ (ítems (c) y (d)) cuando $\delta_j^{[l]} \approx 0$?

**Ejercicio 12.9.** Para cada problema indicar: cantidad de neuronas de salida, función de activación de la última capa, función de pérdida y cómo se obtiene la predicción final (argmax, umbral, etc.)

(a) Reconocer dígitos escritos a mano (0–9).

(b) Detectar si un mail es spam.

(c) Asignarle géneros a una película (acción, comedia, drama, terror, romance). Una película puede tener varios.

(d) Clasificar un tweet como positivo, neutro o negativo.

(e) Diagnosticar a partir de una radiografía la presencia de 5 patologías. Un paciente puede tener varias o ninguna.

(f) Predecir el precio de una casa.

Para el último caso (regresión): ¿cuándo tiene sentido utilizar una función de activación lineal en la última capa? ¿Y una ReLU?

**Ejercicio 12.10.** Verdadero o Falso (justificar en cada caso):

(a) En un problema *multilabel* con activación sigmoidea en la última capa, las salidas $\hat{y}_k$ siempre suman 1.

(b) En un problema multiclase, una vez entrenada la red, para obtener la clase predicha da lo mismo aplicar sigmoidea + argmax que softmax + argmax (o directamente argmax sobre los *logits*), porque la predicción es la misma.

(c) En un problema *multilabel* con $C$ etiquetas, entrenar una red con capas ocultas y $C$ neuronas de salida es equivalente a entrenar $C$ redes binarias independientes (una por cada etiqueta).

**Ejercicio 12.11. Entropía cruzada como función de pérdida**
Una red clasifica imágenes en 3 clases (*perro*, *gato*, *pato*) y termina con una capa *softmax*: para cada instancia devuelve $\hat{\mathbf{y}}^{(i)} = (\hat{y}^{(i)}_{\text{perro}}, \hat{y}^{(i)}_{\text{gato}}, \hat{y}^{(i)}_{\text{pato}})$, con $\hat{y}_c^{(i)} = P(Y = c \mid X = \mathbf{x}^{(i)})$. Con la etiqueta $\mathbf{y}^{(i)}$ en formato *one-hot*, la entropía cruzada de una instancia y la del conjunto son

$$\mathrm{CE}^{(i)} = -\sum_c y_c^{(i)} \ln \hat{y}_c^{(i)} \qquad\qquad \mathrm{CE}_{X,y} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{CE}^{(i)}$$

Dos modelos, $A$ y $B$, predicen sobre las mismas 4 instancias:

| $i$ | clase real | $\hat{\mathbf{y}}^{(i)}$ del modelo $A$ | $\hat{\mathbf{y}}^{(i)}$ del modelo $B$ |
|---|---|---|---|
| 1 | perro | $(0.90,\ 0.05,\ 0.05)$ | $(0.50,\ 0.30,\ 0.20)$ |
| 2 | gato | $(0.10,\ 0.80,\ 0.10)$ | $(0.30,\ 0.50,\ 0.20)$ |
| 3 | pato | $(0.90,\ 0.09,\ 0.01)$ | $(0.50,\ 0.10,\ 0.40)$ |
| 4 | perro | $(0.70,\ 0.20,\ 0.10)$ | $(0.60,\ 0.20,\ 0.20)$ |

| $p$ | $-\ln p$ |
|---|---|
| 0.9 | 0.11 |
| 0.8 | 0.22 |
| 0.7 | 0.36 |
| 0.6 | 0.51 |
| 0.5 | 0.69 |
| 0.4 | 0.92 |
| 1/3 | 1.10 |
| 0.01 | 4.61 |

(a) Calcular el *accuracy* y la $\mathrm{CE}_{X,y}$ de cada modelo. ¿Cuál es preferible y por qué?

(b) Como $\mathbf{y}^{(i)}$ es *one-hot*, en la sumatoria de $\mathrm{CE}^{(i)}$ sólo sobrevive el término de la clase real de la instancia, $r_i$, quedando $\mathrm{CE}^{(i)} = -\ln \hat{y}^{(i)}_{r_i}$. Graficar esquemáticamente $-\ln p$ para $p \in (0, 1]$ y explicar a partir del gráfico por qué se dice que la CE mide la *sorpresa* del modelo al ver la etiqueta real.

(c) En la instancia 3, ¿qué le pasa a la CE del modelo $A$ si la probabilidad de *pato* tiende a 0? ¿Y al *accuracy*? Comparar con el error cuadrático sobre el vector de probabilidades, $\sum_c (\hat{y}_c^{(i)} - y_c^{(i)})^2$: calcularlo para la instancia 3 en ambos modelos y decir cuánto puede valer como máximo.

(d) Un modelo que no aprendió nada predice siempre $(1/3,\ 1/3,\ 1/3)$. Calcular su CE y compararla con la de $A$ y $B$. ¿Qué dice esto del modelo $A$, a pesar de su *accuracy*?

(e) Mostrar que minimizar $\mathrm{CE}_{X,y}$ equivale a maximizar $\prod_i \hat{y}_{r_i}^{(i)}$, la probabilidad que el modelo le asigna a *todas* las etiquetas observadas (suponiendo instancias independientes). Verificarlo con los números del modelo $B$.

(f) **Gradientes.** En el caso binario, con $\hat{y} = \sigma(z)$ y $z$ la salida de la última neurona antes de la sigmoidea, mostrar que

$$\frac{\partial}{\partial z}\Big[ -y \ln \hat{y} - (1-y)\ln(1-\hat{y})\Big] = \hat{y} - y \qquad \text{mientras que} \qquad \frac{\partial}{\partial z}(\hat{y}-y)^2 = 2(\hat{y}-y)\,\hat{y}\,(1-\hat{y}).$$

Evaluar ambas derivadas para una instancia con $y = 1$ y $z = -5$ ($\hat{y} \approx 0.007$). ¿Con cuál de las dos pérdidas aprende más rápido la red cuando se equivoca con mucha confianza? ¿Qué relación tiene $\hat{y} - y$ con el $\delta^{[L]}$ del Ejercicio 12.8?

**Ejercicio 12.12. Softmax y entropía cruzada juntas**
La última capa de un clasificador en $C$ clases calcula los *logits* $\mathbf{z} = (z_1, \ldots, z_C)$ y les aplica softmax:

$$\hat{y}_c = \mathrm{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}}$$

La pérdida de una instancia cuya clase real es $r$ es $\mathrm{CE} = -\ln \hat{y}_r$.

(a) Mostrar que el logaritmo de la CE "deshace" la exponencial del numerador de softmax:

$$\mathrm{CE} = -z_r + \ln \sum_{k=1}^{C} e^{z_k}$$

(b) A partir de esa expresión, mostrar que $\dfrac{\partial\, \mathrm{CE}}{\partial z_c} = \hat{y}_c - y_c$, con $\mathbf{y}$ la etiqueta *one-hot*. ¿Entre qué valores está cada componente del gradiente? ¿Cuándo es cercano a cero?

(c) Para las clases (*perro*, *gato*, *pato*), los logits $\mathbf{z} = (3,\ -2,\ 0.5)$ y la clase real *gato*, calcular $\hat{\mathbf{y}}$, la CE (de las dos formas) y el gradiente respecto de $\mathbf{z}$. Dato: $e^3 + e^{-2} + e^{0.5} \approx 21.87$ y $\ln 21.87 \approx 3.09$.

(d) **Estabilidad numérica.** En punto flotante de 64 bits, el mayor número representable es $\approx 1.8 \cdot 10^{308} \approx e^{709}$ y el menor positivo es $\approx 5 \cdot 10^{-324} \approx e^{-744}$: un resultado más chico se redondea a $0$. Con logits $\mathbf{z} = (1000,\ 0,\ -1000)$ y clase real *gato*, calcular la CE de estas tres formas e indicar qué devuelve cada una:

1. softmax directa, $\hat{y}_c = e^{z_c} / \sum_k e^{z_k}$ (por ejemplo, `torch.exp(z) / torch.exp(z).sum()`), y después $-\ln \hat{y}_r$;
2. softmax restando el máximo $m = \max_k z_k$, $\hat{y}_c = e^{z_c - m} / \sum_k e^{z_k - m}$, y después $-\ln \hat{y}_r$;
3. la expresión del ítem (a), mostrando que $\ln \sum_k e^{z_k} = m + \ln \sum_k e^{z_k - m}$.

¿Qué problema resuelve restar el máximo y cuál no?

(e) Un alumno implementa en PyTorch un clasificador de 3 clases cuya última capa es `nn.Softmax()`, y lo entrena minimizando `nn.CrossEntropyLoss()`, que aplica internamente a su entrada la fórmula del ítem (a) con el truco del máximo de (d). ¿Qué función está minimizando realmente? ¿Cuál es la mínima pérdida que puede alcanzar en una instancia, aunque la clasifique bien con total confianza? ¿Cómo debería terminar la red?